COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	October 7, 2026	**DATE SUBMITTED:**	October 9 , 2026
PAIR MEMBERS:	Thomas Joseph T. Astudillo Johrel Louie T. Silao	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11.1 Basic Algorithm Analysis

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y) {
        for(int i = 0; i < y.length; i++) {
                if(search(x, y[i])!= - 1) {
                return false;
                }
        }
        return true;
}
```

Performing worst case analysis:

Let m = the length of array x.
Let n = the length of array y.

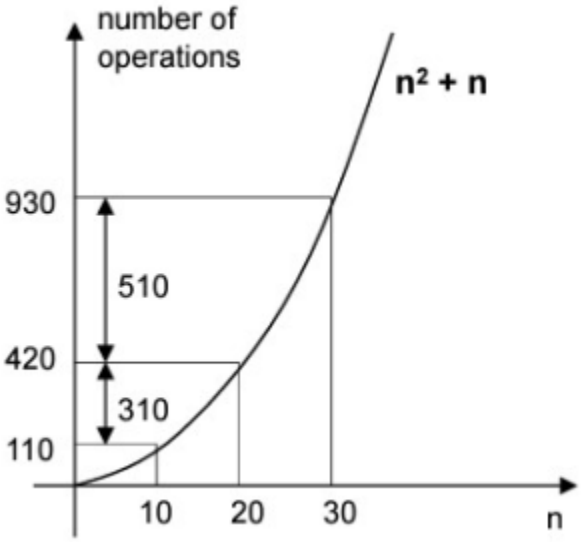
The loop in diff repeats n times.

Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: $O(n^2)$

Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis of Astudillo:

- As the array size increases, the total number of operations also increases, as the graph illustrates the non-linear growth of the time complexity of $O(n^2)$. An example of this is the array size 10, where it uses the quadratic equation of $n^2 + n$, where n is the size of the array, equating to $10^2 + 10$, which is equal to 110. When the array size doubles to 20, the number of operations equals to 420 due to the equation $20^2 + 20$, which adds 310 operations. Finally, when the array size triples to 30, the total operations increases drastically to 930 due to the equation $30^2 + 30 = 930$, adding another 510 operations. This accelerating growth demonstrates that as input sizes scale, the computational cost increases drastically, making the algorithm less efficient for larger datasets.

Analysis of Silao:

- Whenever the array size increases, the total number of operations also increases, this is due to every iteration of 'n' in a loop the 'diff' invokes 'search', which will perform 'm' comparisons to check every array of 'x' without finding a match.The graph shows the quadratic model of the total operations and the non-linear shape of the curves shows how fast the computation increases. This accelerating rate of increase proves how a significant performance degradation characteristic of $O(n^2)$ worst case time complexity when computing larger datasets.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Table 1.1: Analyzing results of runtime performance by comparing experimental and theoretical analysis

Code Screenshot

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <cmath>

class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    // Constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    // Getters
    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }

    // Two people are same if they have the same name
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }

    // The ordering of a set of people is defined by their name
    bool operator<(const Student& p) const {
        return this->name < p.name;
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};

auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
    // The IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}

// Forward declaration
bool needs_vaccination(Student P, std::vector<Student>& people);

void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++) {
        people.push_back(generate_random_Student(size));
    }
    std::sort(people.begin(), people.end());
```

```cpp
    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    if (search_result) {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") needs vaccination." << std::endl;
    } else {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") does not need vaccination." << std::endl;
    }
}

bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (true) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

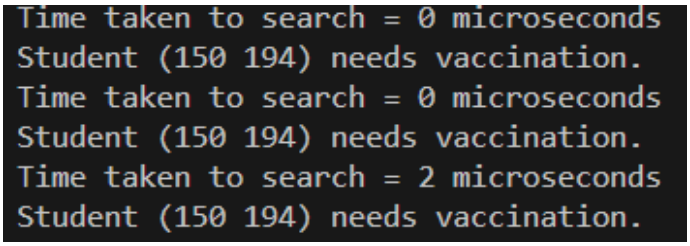
        // Return true if the Student is found in the sequence and he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false) {
            return true;
        } else if (mid_element == P && mid_element.is_vaccinated() == true) {
            return false;
        } else if (mid_element > P) {
            std::advance(last, -mid_element_index);
        } else if (mid_element < P) {
            std::advance(first, mid_element_index);
        }

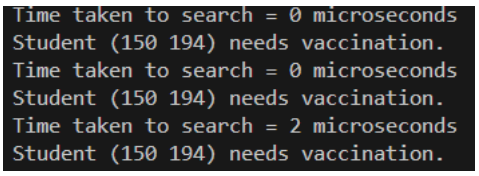
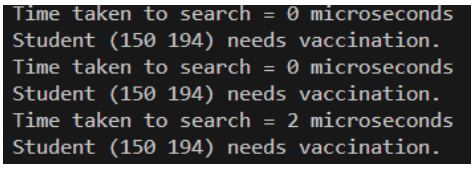
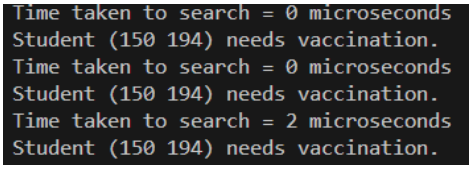
        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1) {
            return true;
        }
    }
}

int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

Astudillo Output

```
Time taken to search = 1 microseconds
Student (118 349) needs vaccination.
Time taken to search = 1 microseconds
Student (118 349) needs vaccination.
Time taken to search = 3 microseconds
Student (118 349) needs vaccination.
```

Silao Output	
Observation of Astudillo	This program utilizes a binary search algorithm using the function 'needs_vaccination' in order to identify if the target student in the vector is vaccinated. If the target student is found and is already vaccinated, it returns false. However, if they are found but unvaccinated it returns true in order to indicate that they need vaccination. Then, the 'search_test' along with the 'main' function tests the needs_vaccination operation across 1,000, 10,000 and 100,000 students to test how long the search takes.
Observation of Silao	This program shows how a binary search algorithm is used to check a student's vaccination status. This demonstrates the O(log N) time efficiency where it increases the number of search comparisons, which evaluates to a large increase in execution time.

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	1 microseconds		The program executes logarithmic comparisons which results in a microsecond execution time.
10,000	1 microseconds		Increasing the input size only increases the binary search steps which results in a minimal change in execution time.
100,000	3 microseconds		Huge increase of input size which results in a big difference in execution time, demonstrating the O(log n).

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.
Algorithm Unique(A)
for i = 0 to n-1 do
for j = i+1 to n-1 do
if A[i] equals to A[j] then
return false
return true

Table 2.1

C/C++

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <cmath>

class UniqueArray {
private:
    std::vector<int> array;

public:
    // Constructor
    UniqueArray(const std::vector<int>& arr) : array(arr) {}

    // Check if all elements in the array are unique
    bool Unique_Array() const;

    // Checking the Array and displaying its contents
    static void checkArray(int size);
};

bool UniqueArray::Unique_Array() const {
    int n = array.size();
```

```
    // Check for duplicate elements in the array
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            // Return False if a duplicate element is found in the array
            if (array[i] == array[j]) {
                return false;
            }
        }

        // Creates a copy of the array and sorts the array
        std::vector<int> sorted_arr = array;
        std::sort(sorted_arr.begin(), sorted_arr.end());
        // Checks adjacent elements to identify whether they are a duplicate
        for (size_t i = 0; i < sorted_arr.size() - 1; i++) {
            // Return false if a duplicate element is found
            if (sorted_arr[i] == sorted_arr[i + 1]) {
                return false;
            }
        }
        return true;
    }

    // Return true if the array is empty or no duplicates were detected
    return true;
}


void UniqueArray::checkArray(int size) {
    // Add elements in the the array
    std::vector<int> Array(size);
    for (int i = 0; i < size; i++) {
        Array[i] = i;
    }

    UniqueArray checker(Array);

    // Measures the starting time
    auto begin = std::chrono::steady_clock::now();
    // Calls the Unique_Array function
    bool result = checker.Unique_Array();
    // Ends the timer
    auto end = std::chrono::steady_clock::now();

    // Calculates the time to search using microseconds
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end -
begin).count();
    // Display the data that were recorded in the begin - end timer
```

```
    std::cout << "Array Size: " << size
              << " | Time: " << duration << " microseconds"
              << " \nUnique: " << (result ? "true" : "false") << std::endl;
}

int main () {
    // Testing Values
    UniqueArray::checkArray(1000);
    UniqueArray::checkArray(10000);
    UniqueArray::checkArray(100000);

    return 0;
}
```

Terminal Output

```
Array Size: 1000 | Time: 143 microseconds
Unique: true
Array Size: 10000 | Time: 2081 microseconds
Unique: true
Array Size: 100000 | Time: 22307 microseconds
Unique: true
```

Observations

Observation of Astudillo	**Theoretical analysis** This uses nested loops to search an array for duplicate values. The worst-case scenario occurs when every element in the array is unique or a duplicate value appears at the final pair of positions. In the best-case scenario, this will stop right away after one comparison if the first two elements are the same. **Experimental analysis** The algorithm's execution time closely matches the given data when tested experimentally with distinct array datasets. However, as the input size increases, the runtime it takes in order to check if it's unique also increases exponentially.
Observation of Silao	**Theoretical analysis** This checks for duplicate values in an array using nested loops. The best case scenario this will stop immediately after one comparison if the first two elements are the same while the worst case scenario happens when all elements in the array are unique or a duplicated value appears at the last pair of positions. **Experimental analysis**

	When checking the algorithm experimentally using unique array datasets, the execution time aligns closely with the mathematical predictions. However, with an increased input size this will cause the runtime to jump largely.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

Algorithm rpower(int x, int n):
1. if n == 0 return 1
2. else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1. if n == 0 return 1
2. if n is odd then
3. y = brpower(x, (n-1)/2)
4. return x*y*y
5. if n is even then
6. y = brpower(x, n/2)
7. return y*y

Table 2.2

C/C++

```
#include <iostream>
#include <chrono>

// This function uses a Linear power function
// Uses the time complexity of O(n)
long long rpower(int x, int n) {
    long long result = 1;
    for (int i = 0; i < n; ++i) {
        result *= x;
    }
    return result;
}

// This function uses a binary power function
// Uses the time complexity of O(log n)
long long brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) {
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
```

```
        long long y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    int x = 2, n = 100000;

    // Measures the starting time for rpower
    auto start1 = std::chrono::high_resolution_clock::now();
    // Calls the linear power function
    rpower(x, n);
    // Ends the timer for rpower
    auto end1 = std::chrono::high_resolution_clock::now();

    // Measures the starting time for brpower
    auto start2 = std::chrono::high_resolution_clock::now();
    // Calls the binary power function
    brpower(x, n);
    // Ends the timer for brpower
    auto end2 = std::chrono::high_resolution_clock::now();

    // Display the data acquired during the rpower and brpower in microseconds
    std::cout << "rpower  (O(n)): "
              << std::chrono::duration_cast<std::chrono::microseconds>(end1 -
start1).count() << " microseconds\n";
    std::cout << "brpower (O(log n)): "
              << std::chrono::duration_cast<std::chrono::microseconds>(end2 -
start2).count() << " microseconds\n";
    return 0;
}
```

Terminal Output

```
rpower  (O(n)): 246 microseconds
brpower (O(log n)): 0 microseconds
```

Observations

Observation of Astudillo	**Theoretical analysis** The code utilizes a linear and binary recursion in order to solve powers by decrementing the exponent at each step in the program's runtime. This results in a n + 1 total recursive call, which in turn gives a linear time complexity of O(n)

	Experimental analysis Having a larger exponent displays a significant difference between the two approaches. The rpower algorithm executes approximately 1,000,000 calls while the brpower solves the same dataset but in only 20 recursive calls.
Observation of Silao	**Theoretical analysis** This utilizes linear recursion to solve powers by decrementing the exponent at each step. This results in an n + 1 total recursive which gives a linear time complexity of O(n) **Experimental analysis)** Having a large exponent shows a huge difference between two approaches. The rpower algorithm executes 1,000,000 recursive calls while brpower solves the same power calculation in only 20 recursive calls.

Conclusion (Astudillo)
Based on the laboratory today, I was able to analyze, implement and utilize various algorithm analysis methods in order to measure and compare the program's runtimes in microseconds across various dataset sizes. Through the intended learning outcomes, I learned how linear logic such as O(n) compares to optimized binary logic such as O (log n), as well as how nested search operations scale up to quadratic time complexity. Utilizing these basic algorithm analysis methods, I was able to create two programs that checks if the array is unique and how long the program takes in order to check if it's unique, as well as a linear and binary power function to check how long it takes for the time complexity to calculate the results. Overall I believe I did quite well along with my partner in order to answer the intended learning outcomes and supplementary activities that were given to us. Although we had a difficult time in doing the supplementary activities at first, we simply just used the initial code as a basis for our code in order to create two programs that checks a unique array and checks a linear and binary power function's time complexity on a dataset. This activity helped me expand my knowledge regarding the very basics of algorithm analysis and how I can apply them to future programs as a future computer engineering student.
Conclusion (Silao)
In this hands-on activity, I was able to learn how to measure the runtime of algorithms using theoretical analysis. Additionally, I was able to analyze the results of runtime performance by comparing experimental and theoretical analysis. All in all, doing this activity wasn't easy but in the end, I was able to finish this with my collaborator with ease.

"I affirm that we will not give or receive any unauthorized help on this activity and that all work will be our own."

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Quillbot, Gemini AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Analysis & Grammar Checking**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We utilized Gemini AI in order to identify how the code in the Intended Learning Outcome B functions in order to implement it for our supplementary activities, we simply used this as a basis for our code and we implemented our own knowledge on how to efficiently code it based on the given problem or pseudo-code. We also used Quillbot in order to check the grammar on each of our explanations and analysis to ensure that the message is delivered clearly.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.